

Session 2.1: Web Data Extraction: Scraping Financial and Competitor Intelligence with BeautifulSoup

1:2 DELIVERY RHYTHM

Applied Learning

60 minutes of focused instruction followed by 120 minutes of guided coding and debugging.

SESSION OUTCOME

Operational Competency

Students apply the session concept to a small, verifiable business dataset.

Concepts for This Session

CONCEPT 1

HyperText Transfer Protocol (HTTP GET)

The standard communication protocol used by web browsers and Python scripts to request public web page markup from external web servers.

CONCEPT 2

Document Object Model (DOM Tree)

The hierarchical tree structure representing an HTML web page, where parent tags contain nested child elements, classes, and text attributes.

CONCEPT 3

CSS Selector Targeting (.select() / .find())

Directing Python to extract specific information from a web page by matching tag names (<table>), CSS classes (.price-tag), or element identifiers (#product-grid).

CONCEPT 4

Beautiful Soup Parser

A Python library (bs4) that navigates, searches, and modifies the Document Object Model tree to extract clean textual data.

How the Concept Works in Practice

FOUNDATION 1

Translating Web Markup into Tabular Intelligence

Most external competitor intelligence (pricing, product assortment, inventory availability) is published on public HTML pages rather than clean CSV files. Web scraping automates the extraction of this unstructured markup into structured tabular datasets.

FOUNDATION 2

Navigating the HTML Document Object Model

Understanding the tree hierarchy of web documents. An e-commerce product catalog consists of repeated container elements (`<div class="product-card">`) housing title headings, price tags, and inventory spans.

FOUNDATION 3

Robust Selector Strategy

Target semantic CSS classes and attributes rather than fragile absolute XPath coordinates. If a website changes its navigation bar, robust CSS class selectors continue extracting pricing tables without breaking.

FOUNDATION 4

Ethical Scraping and Rate Limiting

When sending automated HTTP requests, scripts must incorporate random pauses (`time.sleep(2)`) to avoid overwhelming public servers, preventing IP blocking and maintaining compliance with commercial terms of service.

Build, Test, and Diagnose in the IDE

[1] FILE EXPLORER

Organize Assets

Use the session dataset and keep scripts in a clear workspace.

[2] CODE EDITOR

Write Small Segments

Run each completed segment before expanding the program.

[3] AI AGENT CHAT

Diagnose Errors

Capture the traceback, request an explanation, and review the fix.

[4] INTEGRATED TERMINAL

Verify Execution

Confirm the final program completes with return code 0.

Script Demonstration 1: Web Data Extraction: Scraping Financial and Competitor Intelligence with BeautifulSoup

```
import requests
from bs4 import BeautifulSoup

# Simulated public e-commerce pricing table
html_markup = """
<div class="catalog">
  <div class="product" data-sku="SKU-801">
    <h3 class="name">Enterprise Router</h3>
    <span class="price">$299.99</span>
    <span class="stock">In Stock (45 units)</span>
  </div>
  <div class="product" data-sku="SKU-802">
    <h3 class="name">Office Desk Chair</h3>
    <span class="price">$49.50</span>
    <span class="stock">Low Stock (8 units)</span>
  </div>
</div>
"""

soup = BeautifulSoup(html_markup, "html.parser")
print("Document Title Extracted:", soup.find("h3").text)
```

SYNTAX

Readable Structure

Observe the syntax and indentation that control execution.

RUNTIME

Verified Execution

Run the example in the integrated terminal and inspect the output.

BUSINESS VALUE

Applied Automation

Use the pattern to automate a repeatable business decision.

Script Demonstration 2: Web Data Extraction: Scraping Financial and Competitor Intelligence with BeautifulSoup

```
products = []
for item in soup.select(".product"):
    sku = item["data-sku"]
    name = item.select_one(".name").text.strip()
    raw_price = item.select_one(".price").text.strip()
    price_usd = float(raw_price.replace("$", ""))
    products.append({"sku": sku, "name": name, "price_usd":
price_usd})

print("Parsed Products:", products)
```

SYNTAX

Readable Structure

Observe the syntax and indentation that control execution.

RUNTIME

Verified Execution

Run the example in the integrated terminal and inspect the output.

BUSINESS VALUE

Applied Automation

Use the pattern to automate a repeatable business decision.

Script Demonstration 3: Web Data Extraction: Scraping Financial and Competitor Intelligence with BeautifulSoup

```
import pandas as pd

catalog_df = pd.DataFrame(products)
print(catalog_df.describe())
```

SYNTAX

Readable Structure

Observe the syntax and indentation that control execution.

RUNTIME

Verified Execution

Run the example in the integrated terminal and inspect the output.

BUSINESS VALUE

Applied Automation

Use the pattern to automate a repeatable business decision.

Exercise 1 and Exercise 2

EXERCISE 1: GUIDED

Foundational Practice

Follow the lab protocol, run the script, and verify its output with the session dataset.

EXERCISE 2: DEBUGGING

Applied Challenge

Extend the program, diagnose an intentional error, apply the verified correction, and confirm return code 0.